

國立陽明交通大學
資訊科學與工程研究所
碩士論文

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University
Master's Thesis

Compile Once, Execute Many：重複性模板實例化任務
的可驗證規格編譯方法

Compile Once, Execute Many: Verifiable Specification
Compilation for Recurrent Template-Instantiation Tasks

研 究 生： 林冠丞 (Lin, Kuan-Cheng)
指導教授： 陳添福 (Chen, Tien-Fu)

中華民國 一一五年十一月
November 2026

Compile Once, Execute Many：重複性模板實例化任務的
可驗證規格編譯方法

Compile Once, Execute Many: Verifiable Specification
Compilation for Recurrent Template-Instantiation Tasks

研 究 生： 林冠丞

Student： Kuan-Cheng Lin

指導教授： 陳添福 博士

Advisor： Dr. Tien-Fu Chen



November 2026
Taiwan, Republic of China

中華民國 一一五年十一月

誌 謝

(致謝待撰寫。)

林冠丞 於

國立陽明交通大學 資訊科學與工程研究所

中華民國 一一五年十一月



Compile Once, Execute Many：重複性模板實例化任務的可驗證規格編譯方法

學生：林冠丞

指導教授：陳添福 博士

國立陽明交通大學
資訊科學與工程研究所

摘 要

(中文摘要待撰寫。)

關鍵字：(待定，5-7 個)



Compile Once, Execute Many: Verifiable Specification Compilation for Recurrent Template-Instantiation Tasks

Student : Kuan-Cheng Lin

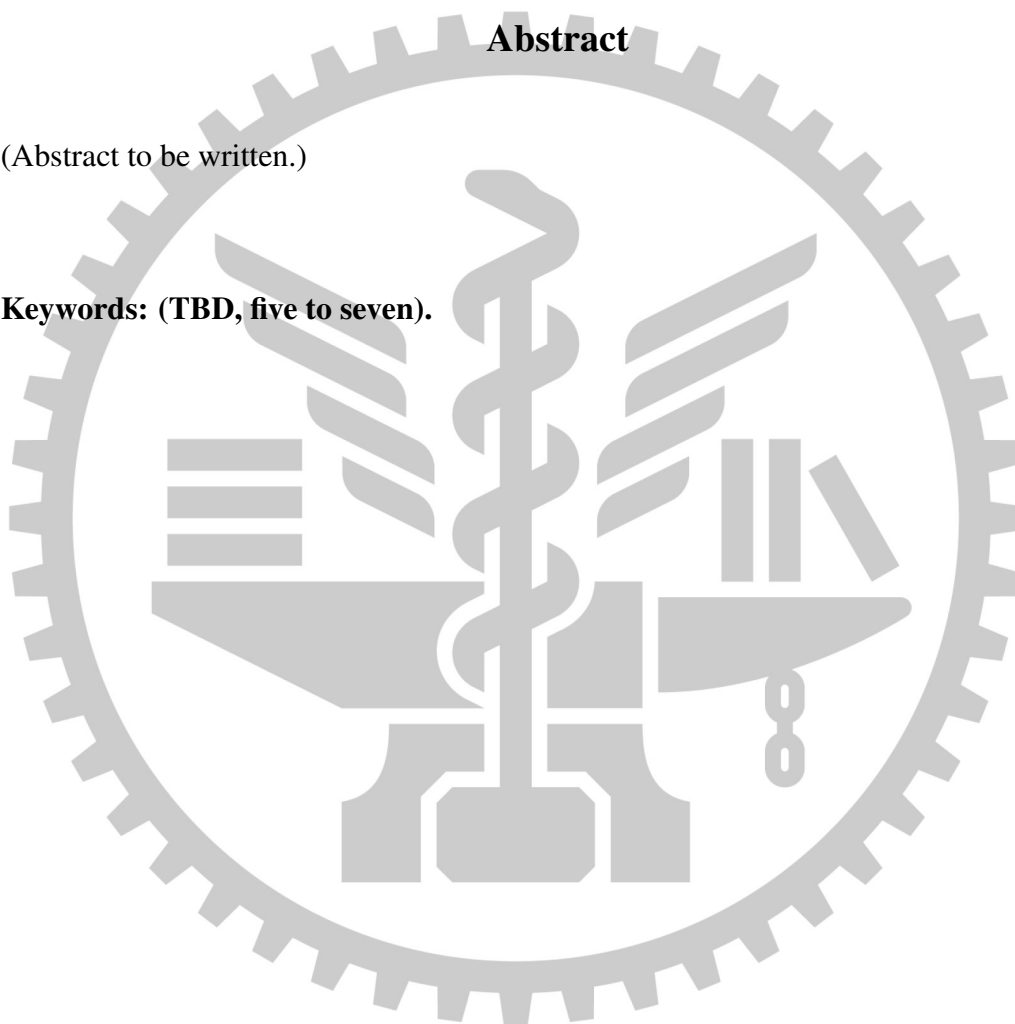
Advisor: Dr. Tien-Fu Chen

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University

Abstract

(Abstract to be written.)

Keywords: (TBD, five to seven).



Contents

Chinese Abstract	i
English Abstract	ii
Contents	iii
List of Figures	iv
List of Tables	v
1 Introduction	1
1.1 Motivation	1
1.2 Two Existing Approaches, Both Wrong	1
1.3 Our Position	1
1.4 Contributions	1
1.5 Claims and Where They Are Tested	1
1.6 Thesis Organization	1
2 Related Work	2
2.1 Template-Shaped Tasks	3
2.2 Document Formats and Representation Mismatch	3
2.3 Failure Modes	3
2.4 Governance Requirements	3
2.5 LLMs for Document and Engineering-Drawing Understanding	3
2.6 Document Automation, RPA, and Template Filling	3

2.7	Programming by Example and Program Synthesis	3
2.8	Tool-Making, Skill Libraries, and Amortized LLM Computation	3
2.9	Execution-Feedback Program Repair	3
2.10	Verification and Guardrails for LLM Output	3
2.11	Cost-Aware Agent Evaluation	3
3	System Model	4
3.1	Problem Formulation	4
3.1.1	A Stateful, Two-Stage Input Model	4
3.1.2	Compile-Once, Execute-Many	7
3.1.3	Required Properties of the Specification IR	9
3.1.4	Measured Boundaries of the Specification IR	12
3.1.5	Design Principle: Fail Loud, Not Silent	16
3.1.6	Scope and Applicability Conditions	16
3.2	Domain-Dependent and Domain-Independent Layers	16
3.3	Specification IR	16
3.4	Structured Dump as Grounding	16
3.5	Locator Mechanisms	16
3.5.1	Semantic Locators	16
3.5.2	Range Locators and Source-Side Aggregation	16
3.5.3	Failure Modes and What Each Mechanism Blocks	16
3.6	Target Reference and Write-Back	16
3.7	Deterministic Compiler and Zero-LLM Runtime	16
3.8	Lifecycle Management	16
3.8.1	Lineage and Versioning	16

3.8.2	Regression Suite.....	16
3.8.3	Drift Detection	16
3.8.4	Auditability	16
3.9	Expressiveness Boundary	16
4	Methodology	17
4.1	State Machine.....	17
4.2	Structural Gates.....	20
4.2.1	What a Structural Gate Catches That an Instance Run Cannot	22
4.3	Two Complementary Oracles.....	23
4.3.1	The Author-Time Expected-Text Oracle.....	24
4.3.2	The Case-Level Oracle.....	26
4.3.3	Why Not an LLM Judge	26
4.4	Case-Repair Loop	28
4.4.1	Two Loops, Deliberately Distinct.....	29
4.4.2	Four Invariants	29
4.4.3	A Refusal That Was Correct	30
4.5	Human in the Loop: Evidence Review	31
4.5.1	What Is Recorded.....	31
4.5.2	What Publication Refuses	32
4.6	Coverage Is the Real Bottleneck.....	33
4.6.1	Why the First Denominator Did Not Work.....	34
4.6.2	The External Denominator.....	34
4.6.3	Metric Plumbing Is Methodology	35
4.6.4	What Coverage Does and Does Not Measure.....	36

4.7	Boundaries of Author-Time Verification	37
5	Performance Evaluation.....	42
5.1	Research Questions.....	42
5.2	Study Design.....	44
5.2.1	Why Not a Few-Shot Held-Out Split.....	44
5.2.2	Two-Level Split.....	45
5.2.3	Dataset Construction.....	47
5.3	Baselines and Fairness	47
5.4	Metrics	50
5.4.1	Asymmetric Rerun Budget	50
5.5	The Evaluator.....	50
5.5.1	Separate Channels, Never One Average	50
5.5.2	Two Self-Validations, Neither Sufficient Alone.....	51
5.5.3	A Negative Control for Every Zero	52
5.5.4	The Extent Diagnostic Is Not Symmetric Across Arms	52
5.6	Template Drift.....	54
5.7	Ablations.....	54
5.8	Results.....	54
5.8.1	Where the Randomness Went.....	54
5.9	Case Study: Engineering Drawings	54
5.10	Threats to Validity.....	54
5.10.1	Threats Biased Towards the Proposed Method.....	54
5.10.2	Threats Biased Against the Proposed Method	55
5.10.3	Threats of Undetermined or Mixed Direction	57

6	Conclusions.....	59
6.1	Conclusion	59
6.2	Limitations and When This Method Does Not Apply	59
6.3	Future Work	59
	References.....	60



List of Figures



List of Tables



Chapter 1. Introduction

1.1 Motivation

1.2 Two Existing Approaches, Both Wrong

1.3 Our Position

1.4 Contributions

1.5 Claims and Where They Are Tested

1.6 Thesis Organization



Chapter 2. Related Work

2.1 Template-Shaped Tasks

2.2 Document Formats and Representation Mismatch

2.3 Failure Modes

2.4 Governance Requirements

2.5 LLMs for Document and Engineering-Drawing Understanding

2.6 Document Automation, RPA, and Template Filling

2.7 Programming by Example and Program Synthesis

2.8 Tool-Making, Skill Libraries, and Amortized LLM Computation

2.9 Execution-Feedback Program Repair

2.10 Verification and Guardrails for LLM Output

Chapter 3. System Model

3.1 Problem Formulation

3.1.1 A Stateful, Two-Stage Input Model

The abbreviation $\mathcal{A} : T \rightarrow S$ is potentially misleading because the system does not consume all experimental material in one call, and derivation is neither pure nor deterministic. Let d denote one derivation dataset in template lineage ℓ . We split the available material into a derivation layer and a later regression layer,

$$T = (T_{\text{der},d}^{(n)}, T_{\text{reg},\ell}), \quad (3.1)$$

$$T_{\text{der},d}^{(n)} = (d, \ell, B_d, Y_d^?, \{X_{dj}\}_{j=1}^{N_d}, R_d^?, U_d^?, Q_d, P, \Gamma, \Theta, H_d^{(<n)}), \quad (3.2)$$

$$T_{\text{reg},\ell} = \mathcal{R}_\ell = \{c_i = (I_i, \{X_{ij}\}_{j=1}^{N_i}, u_i, \pi_i, Y_i^?)\}_{i=1}^{K_\ell}. \quad (3.3)$$

Here, B_d is exactly one baseline template; $Y_d^?$ is an optional expected output; $\{X_{dj}\}_{j=1}^{N_d}$ is a possibly empty collection of source artifacts; $R_d^?$ is an optional operational-rule artifact; and $U_d^?$ is an optional inline source-data override. When present, the override bypasses the locally parsed artifact dictionary and, on the agent path, takes precedence over extracted keys. Q_d groups the inspection level, force-refresh choice, content-addressed dump, cache state, CAD preflight and health observations, locator verification, and deterministic validation/repair feedback. The derivation environment is also an input: P denotes the prompt files, Γ the agent's

tool and skill definitions, and Θ the model, gateway, and MCP configuration. Thus, changing a prompt, an allowed tool, or the selected model changes T_{der} even when all customer artifacts are byte-identical.

The term $H_d^{(<n)}$ is the pre-existing state before the n th derive. It contains the field names and rule inventories accumulated from all existing non-degraded specs derived from the same dataset. The implementation reads this state through `derivation_service.DerivationService._field_hints`, and `_previous_rule_inventory`; the degraded flag used by that selection is produced by `cad_health.degradation`. The hints are fed to the agent and the prior inventory is merged into the new spec evidence. With ω_n denoting model sampling and the observed session/tool trace, the honest derivation relation is therefore

$$S_d^{(n)} \sim \mathcal{A}(T_{\text{der},d}^{(n)}; \omega_n), \quad T_{\text{reg},\ell} \notin \text{dom}(\mathcal{A}). \quad (3.4)$$

Consequently, two derives over the same artifact bytes need not have the same context or the same output: $H_d^{(<n)}$ may have changed, and ω_n is stochastic. In this thesis, $\mathcal{A} : T \rightarrow S$ is only shorthand for the projection $\mathcal{A} : T_{\text{der}} \rightsquigarrow S$ in Equation 3.4, not a claim that $\mathcal{A}$ is a pure function.

This split follows the implemented call boundary. The dataset creation and derive paths, `api.create_dataset` and `derivation_service.DerivationService.derive`, admit one baseline, at most one expected artifact, N_d source artifacts, and at most one rules artifact. They do not read `models.DwgTestCase`. Only after S exists does the separate endpoint call `regression.RegressionRunner.run`, which selects every active test case of the spec’s lineage and executes them. K_ℓ is therefore the number of active cases at regression time; the implementation imposes no $K_\ell = 3$ limit. Each case c_i contains an input template I_i , its own source artifacts and explicit source-data override u_i , runtime parameters π_i , and an optional expected output $Y_i^?$.

The prompt, tools, and model terms above are concrete inputs rather than abstract nuisance variables. `derivation_agent.OpenCodeAgent.derive_rules` selects the `dwg-derivation` agent and `OpenCodeAgent._run` submits the task payload; the selected agent is configured by `opencode.jsonc`, its prompt is `prompts/derivation.md`, and its permitted CAD MCP tools are defined by the agent configuration and `mcp_server.server`. These files and settings must therefore be versioned with the material definition used in an experiment.

We reason about the resulting specification as

$$S = (G, L_s, L_t, F, \Pi, \mathcal{E}_S), \quad (3.5)$$

where G contains value-source and transformation semantics, L_s contains source locators, L_t contains independently derived target locators, F contains rendering rules, Π is the runtime-parameter schema, and $\mathcal{E}_S$ is derivation and validation evidence. The persisted `DwgSpec` stores these concerns across `field_map`, `rule projections`, `runtime_params`, and `evidence`; this tuple is a conceptual separation rather than a claim that each component is a distinct database column.

Information boundary. The operational-rule artifact R_d may identify source fields and state formulas, conditions, constants, and formatting requirements, but it must not contain target cell addresses, bookmark names, content-control tags, or mappings of the form `source_field` $\rightarrow$ `target_cell`. Target references belong to L_t and must be derived from the template by $\mathcal{A}$; otherwise R_d becomes an answer table and the derivation problem is leaked into its input. **Status: normative/aspirational only.** The boundary is stated but not enforced, in two independent ways. First, no admission check exists: neither `api.create_dataset`

nor `derivation_service.DerivationService._load_rules` rejects an answer-bearing rules artifact, so a hand-authored R_d containing target addresses is accepted silently. Second, the persisted projection is not structurally safe: `derivation_service.DerivationService` copies the field's target-reference slot into `DwgSpec.mapping_rules`. That slot is carrier-agnostic — its interpretation follows the target kind of the template, so it holds bookmark identities for `.docx` and cell addresses for `.xlsx`, which are exactly the two artefact classes this boundary forbids. (The field retains the legacy name `handles` from the drawing-oriented prototype; the naming is historical and does not narrow the scope.) If that projection is exported, presented as the rule table, or fed back as R_d , it exposes target answers, and $\mathcal{A}$ is then measured on its ability to copy them rather than to derive them. It must therefore remain internal, and be stripped of target references or split from the semantic rules, before the boundary may be described as a property of the system rather than a requirement on it.

3.1.2 Compile-Once, Execute-Many

The target problem class consists of recurrent template-instantiation tasks for which the output is deterministically derivable from structured source data and explicit operational rules. “Compile once” means that the expensive, stochastic derivation in Equation 3.4 produces one reusable S ; it does not mean that a single fixed output job is reused, because instance values differ. For instance input $V_i = (I_i, \{X_{ij}\}, u_i, \pi_i)$, execution deterministically instantiates and

applies the spec,

$$(J_i, P_i) = \mathcal{C}(S, V_i), \quad (3.6)$$

$$D_i = \mathcal{X}(J_i, I_i), \quad (3.7)$$

$$\text{ok}_i = \mathcal{O}_{\text{plan}}(P_i, I_i, D_i) \wedge \begin{cases} \mathcal{O}_{\text{expected}}(D_i, Y_i), & Y_i^? = Y_i, \\ \text{true}, & Y_i^? \text{ is absent.} \end{cases} \quad (3.8)$$

$\mathcal{C}$ is the deterministic per-instance compiler, P_i is its authoritative write plan, $\mathcal{X}$ is the format-specific executor, and D_i is the produced document. The problem is to derive an S once such that ok_i holds for every admissible instance, while the cost of derivation is amortized over repeated executions.

The deterministic core is implementation-backed. In `compiler.compile_cad_job`, the spec, source data, runtime parameters, and current template values are explicit inputs; failures are returned before submission, and the canonical job JSON is content-hashed. The DOCX/XLSX path shares this compiler and applies P_i through `docx_target.DocxTargetAdapter` or `xlsx_target.XlsxTargetAdapter.apply`; `document_production.document_job` records a canonical hash for the document job. Regression is also implementation-backed: `regression.RegressionRunner._run_case` compiles each case, and `regression.RegressionRunner.verify_output` uses `verification.verify_output` plus `verification.compare_expected` when Y_i exists.

This formulation deliberately localizes randomness to $\mathcal{A}$, and the execution path now matches that localization. An earlier revision of this chapter recorded that it did not: `production.ProductionCompiler` invoked `derivation_agent.OpenCodeAgent.repair_cad_job` after a failed gateway job validation, and the comparison that guarded the repaired job excluded target locations,

so a model-relocated target could be accepted and submitted inside the same run. That branch has since been removed from execution. A compiled job the tenant manifest rejects terminates the run with `CAD_JOB_MANIFEST_REJECTED` and an instruction to re-derive, review, and republish; a cached target reference the current template no longer contains terminates it with `CACHED_HANDLE_NOT_FOUND`. Neither is repaired in place, and `repair_cad_job` has no remaining caller on the execution path.

Status: implementation-backed, and pinned by a negative control rather than by inspection. `test_successful_production_operation_records_zero_llm_calls` replaces both agent repair entry points with a recorder that returns a failure, runs a complete production operation, and then asserts both that the recorder was never invoked and that the `LlmOperation` persisted for the run holds zero model calls. Reading the source would only show that the call site is absent today; the test also fails if a future change reintroduces one. This is the evidence behind claim C4, and it is deliberately the narrowest of the thesis’s claims: it covers $\mathcal{X}$, not $\mathcal{A}$, and not the derivation-time repair loop of Chapter 4, which is by construction inside the compile step whose cost repeated execution amortizes.

3.1.3 Required Properties of the Specification IR

The reusable IR must satisfy the following five properties. Each status below refers to the repository state examined for this thesis, not merely to the intended design.

P1—separation of structure and instance values. S must describe reusable source/target locators, transformations, formatting, and parameter types; a particular instance supplies values through V_i . The only values permitted in S are constants explicitly prescribed by R_d , not values copied from a calibration output. **Status: implementation-backed at the representation interface.** `compiler.compile_cad_job` accepts `spec`, `source_data`,

and `runtime_params` separately, while `value_layer.resolve_field_raw` evaluates the value layer at execution time; `regression.RegressionRunner._run_case` reuses the same spec with each case's inputs. This interface supports reuse, although successful regression—not the type shape alone—is what detects a semantically overfit constant.

P2—explainable and verifiable locators. Every source or target locator must expose what it resolved to and must be checkable against the corresponding artifact; an unresolved or ambiguous locator is evidence, not permission to guess. **Status: implementation-backed.** `source_locator.resolve` returns the resolved sheet, cell, selection, and match count, and `source_resolution.verify_against` checks that the locator reproduces the derivation sample. On the target side, `target_ref.BaseTargetAdapter.resolve` and `describe` provide a common existence and explanation interface. `docx_target.DocxTargetAdapter.resolve` and `xlsx_target.XlsxTargetAdapter.inspect` retain labels, physical locations, positional flags, and ambiguous references, while their `apply` methods return every missed reference.

P3—bounded layout resilience and fail-loud behavior. Semantic identities should survive admissible layout changes, while a change outside the declared tolerance must stop the run rather than trigger a guessed write. This includes the zero-LLM execution requirement: an execution-time model must not relocate a target. **Status: implementation-backed for the fail-loud half; the resilience half is bounded and measured rather than general.** Source locators resolve by sheet/header/section semantics; `docx_target.DocxTargetAdapter.resolve` distinguishes stable content-control/bookmark identities from positional table cells; `xlsx_target.XlsxTargetAdapter.resolve` distinguishes stable defined names from positional coordinates; and both reject duplicate or missing references. The execution branch that previously undermined this property — agent repair of a rejected job inside the run — has been removed, as described in Section 3.1.2, so a

target that cannot be resolved now ends the run with a code instead of being relocated.

What is *not* claimed is the resilience half in general form. Layout tolerance is declared per locator mechanism, and a change outside a mechanism's declared tolerance is detected only insofar as the mechanism can see it. Two measured boundaries are carried forward to Section 3.1.4 rather than glossed here: an unused detail-row cell that no column of a repeat block addresses is not cleared by `unused_slots: "blank"`, so a template reused from a previous reporting period can carry a stale value into a fresh run; and the derivation-time semantic oracle compares only references whose text differs between the baseline and the expected document, so a specification that writes into a cell the expected document leaves blank is structurally invisible to it.

P4—diffability, versioning, and lineage. The IR and its evidence must support reviewable changes, immutable publication, and regression over a template lineage. **Status: implementation-backed for the currently produced handle-backed field-map shape.** `models.DwgSpec` carries a lineage-scoped unique version and template fingerprint; `drift.spec_field_drift` reports added, removed, and attribute-level field changes; `models.DwgProductionVersion.save` prevents mutation of the published spec/hash identity; and `regression.RegressionRunner.run` persists results for all active cases of the lineage. The drift function reads `target.handles` rather than the generic `targets` representation; diffability for a future spec that uses only generic target refs remains normative until that path is implemented.

P5—independently auditable rule coverage. *S* must enumerate which rules and fields it claims to cover, and the denominator must come from the rule artifact independently of the model being scored. **Status: implementation-backed since the coverage-v2 path replaced agent self-enumeration.** The original implementation — `rule_coverage.merge_inventory`

over an agent-supplied `rule_inventory`, unioned across prior derives — did not satisfy P5 and is retained only to read historical specifications. A rule the agent never enumerated was absent from the denominator, so “22 of 22 covered” and “22 of 40 covered” were the same observation.

The active path externalizes the denominator instead. A dataset that carries a rule document must also carry a frozen, versioned rule-applicability manifest; `rule_coverage.parse_rule_manifest` verifies that the manifest names the exact SHA-256 of the rule-document bytes and that its declared document roles match the dataset’s, and `derivation_service.DerivationService._load` refuses the derive with `RULE_MANIFEST_MISSING` before any paid model call, so a run cannot discover after the fact that its release denominator is unauditably. The agent supplies a *disposition* for each externally assigned rule identifier and nothing else: `rule_coverage.score_manifest` discards unknown identifiers, refuses duplicate dispositions, and refuses an assignment whose document roles disagree with the manifest, so the scored agent can neither add, remove, rename, nor move a rule between roles. The resulting partition into `covered`, `unresolved`, `rejected`, and `unmentioned` is mutually exclusive and sums to the frozen applicable set, and `gates.canonical_unresolved` turns every non-covered disposition into a publish gate rather than partial credit. Section 4.6 develops what this measures and what it still does not.

3.1.4 Measured Boundaries of the Specification IR

The five properties above state what the IR must do. This subsection states what it demonstrably does not, because a boundary that is only discovered by a reader running the system is indistinguishable from a defect the author did not notice. Each item below is either pinned by a test or was measured on a real artifact; where an alternative shape was considered and rejected,

the reason is given, since “we did not think of it” and “we rejected it” are different claims.

B1—exclude is a row-content predicate. A range’s `exclude` clause inspects the content of a data row. It does not inherit meaning from a preceding section header, so a sheet whose rows alternate between “current month” and “cumulative” groups under one header cannot be partitioned by `exclude`. The rejected alternative was a domain flag such as `exclude_cumulative`; it was refused because it names one agency’s form inside a carrier-neutral IR. The general shape adopted instead is a group projection (`records: a fixed group_size` with named row roles and per-column `select_offset`), which describes the same layout without naming the domain.

B2—stacked records assume a fixed group size. `records` models “ g physical rows form one logical record” for a constant g . A record whose row count varies with its own data is outside the IR. Both the read side and the write side are implemented, and a specification produced by the agent for the XLSX evaluation template declares `records` with `group_size: 2` and named row roles, so the primitive is exercised by a main evaluation dataset rather than by fixtures alone.

B3—the verification basis for the collection primitives is $n = 1$. `artifact_collection`, `records`, and the equality merge of Section 3.5.2 each have exactly one motivating real workflow. Six independently sourced government workbooks were scanned for a second instance of a repeating multi-row record block and none was found; the nearest candidate was a one-off label merge, not a repeating record. The generalization threshold this thesis sets for a new primitive is therefore not met for these three, and they are presented as carrier-neutral shapes with a single validating workflow rather than as generally validated primitives. The risk this creates is specific and worth naming: it does not appear as a failing test, because the tests were written

from the same single example. It appears as a wrong result on a form nobody has tried.

B4—merge is equality-only and inner. Two resolved row sets can be joined on equal key columns. Range containment, interval overlap, and inequality correspondences are not expressible. The join is deliberately inner: supplying a default for an unmatched key would convert a missing record into a normal-looking number without anyone having authorized the default, which is precisely the silent-failure shape this thesis measures. A specification that needs the filled row must produce it on the source side, where the decision is visible. Merges chain, so three tables can be joined as two binary merges; the agent produced a chained pair unprompted.

B5—the aggregation axis is columns only. Aggregation sums named columns. The criterion is who decides the number of columns: if the template does, naming them is legitimate; if *this instance's data* does, the specification has become instance-specific, and the correct modelling is a long table plus a pivot at write time. A row axis is not offered because a wide table's column names would then be data values, which would disable the author-time static check that a formula only names columns the row set actually has.

B6—a repeat block clears only the cells its own columns address. When a block declares `unused_slots: "blank"`, the cells cleared in an unused slot are the cells that block's columns point at. A cell in the same slot that no column addresses is not cleared, because the IR knows a block's targets only through the strings its columns declare, and “every other cell on this row” is not something it can name. For a pristine template this is exactly right, and every instance of both evaluation datasets starts from a pristine template. For the common practice of reusing last period's completed report as this period's template, it means a stale value can sit on a row the system considers written. The behavior is pinned by a test rather than left to inference, and the rejected workaround—declaring a column whose value is always blank—was refused

because it puts layout knowledge into the value layer and makes the column count grow with the template's width.

B7—one instance is one filled target template. A specification may declare several output documents, but their roles must be distinct; a run that would produce M documents of the same role is refused. Relaxing this is a separate decision and is not claimed here.



3.1.5 Design Principle: Fail Loud, Not Silent

3.1.6 Scope and Applicability Conditions

3.2 Domain-Dependent and Domain-Independent Layers

3.3 Specification IR

3.4 Structured Dump as Grounding

3.5 Locator Mechanisms

3.5.1 Semantic Locators

3.5.2 Range Locators and Source-Side Aggregation

3.5.3 Failure Modes and What Each Mechanism Blocks

3.6 Target Reference and Write-Back

3.7 Deterministic Compiler and Zero-LLM Runtime

3.8 Lifecycle Management

3.8.1 Lineage and Versioning

3.8.2 Regression Suite

3.8.3 Drift Detection

3.8.4 Auditability

Chapter 4. Methodology

Chapter 3 described the specification S and the deterministic execution that consumes it. This chapter describes the procedure that produces S : how a stochastic model proposal is turned into an artifact a deterministic layer has interrogated, what each interrogation can and cannot see, and where a human decision is required rather than optional. The organizing constraint throughout is the thesis’s primary risk measure. A derivation procedure that maximizes the number of fields produced is easy; the procedure below is instead arranged so that every way a specification can be wrong is either detected before publication or is explicitly named as undetectable.

4.1 State Machine

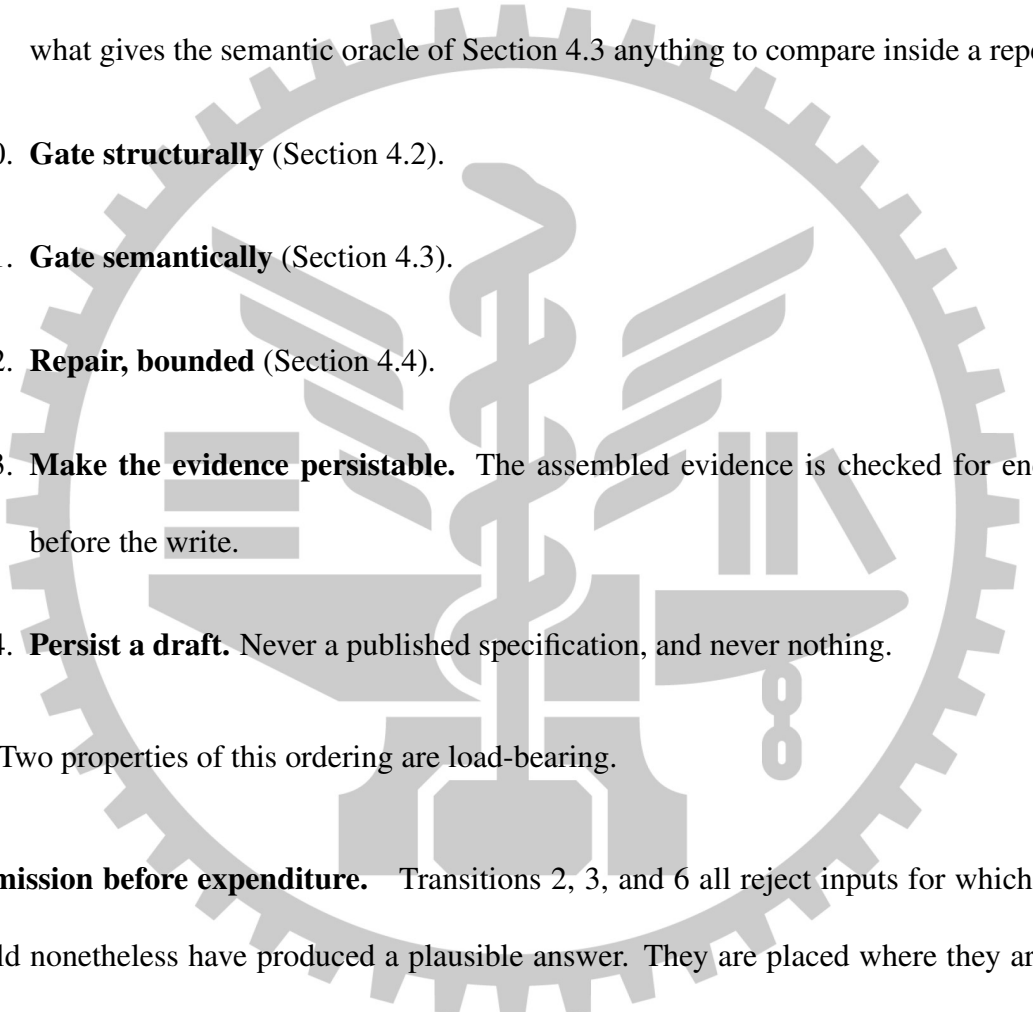
Derivation is a fixed sequence of transitions in which a single stochastic step is surrounded by deterministic admission checks. Writing it as a state machine rather than as “call the agent and validate the result” is not presentation: the order of the transitions is itself a result, because three of them exist only because an earlier ordering wasted a paid model call.

Let Φ denote the transition sequence implemented by `derivation_service.DerivationService`. Given $T_{\text{der},d}^{(n)}$ it produces a persisted draft specification and its evidence, $(S, \mathcal{E}_S)$. The transitions are:

1. **Ground.** Each declared output template is inspected into a structured dump. Dumps are content-addressed and cached, so re-deriving over unchanged bytes does not re-extract

them; the cache disposition per template is reported with the result rather than hidden.

2. **Admit the denominator.** If the dataset carries a rule document it must also carry a frozen rule-applicability manifest. `DerivationService._load_rule_manifest` verifies the manifest against the exact bytes of the rule document and against the dataset's declared document roles, and fails the derive on any mismatch.
3. **Resolve the carrier.** The target kind is computed from the output artifact before the agent is addressed, because the agent is instructed differently for a drawing than for a document, and a `.docx` derive framed as a drawing task is told to call tools that cannot exist for it.
4. **Warm the drawing gateway,** for the DWG carrier only. Probing a CAD gateway for a `.docx` would fail against a perfectly healthy installation.
5. **Propose.** The agent receives the dumps, the staged original source and rule artifacts, the externally assigned rule identifiers, the field-name hints from prior non-degraded specifications of the same dataset, and the declared document roles. This is the only stochastic transition and the only paid one.
6. **Admit the proposal's addressing.** A reply naming a document role the dataset has no template for is refused with `AGENT_DOCUMENT_ROLE_UNKNOWN`. Silently dropping those fields would leave a specification that is merely short, while rule coverage still counted them as covered.
7. **Bind.** The per-document field map, repeat blocks, and shared extents are assembled, and target references the agent did not supply are recorded as unresolved with a cause that distinguishes an infrastructure outage from a rule that was not understood.

- 
8. **Verify the source side.** Every declared source locator is re-resolved against the derivation’s own sample artifacts and must reproduce the value the derivation used; failures and disagreements are recorded, not assumed away.
 9. **Resolve the detail tables.** Named extents are resolved into row sets against the calibration instance’s own workbooks, and their values are folded into the sample data. This costs no model call—it is the same deterministic pre-pass that execution runs—and it is what gives the semantic oracle of Section 4.3 anything to compare inside a repeat block.
 10. **Gate structurally** (Section 4.2).
 11. **Gate semantically** (Section 4.3).
 12. **Repair, bounded** (Section 4.4).
 13. **Make the evidence persistable.** The assembled evidence is checked for encodability before the write.
 14. **Persist a draft.** Never a published specification, and never nothing.
- Two properties of this ordering are load-bearing.

Admission before expenditure. Transitions 2, 3, and 6 all reject inputs for which the agent could nonetheless have produced a plausible answer. They are placed where they are because the alternative was measured: a dataset assembled without a declared output role was rejected in roughly half a second at zero model cost by `RULE_MANIFEST_ROLE_MISMATCH`, whereas the same defect discovered after transition 5 would have discarded the entire paid run. The general rule this encodes is that a check whose inputs are all available before the model call belongs before the model call, even when placing it later would be simpler to write.

Derivation fails to publish, not to produce. No transition after the agent’s reply can cause the derive to return nothing. A specification that will not compile is still persisted, with the reason recorded, because a broken draft a reviewer can read is worth more than an error message. What the procedure must not do is let such a draft look healthy, and that is a separate mechanism: everything that survives unrepaired is written into `evidence.spec_validation`, and `gates.canonical_unresolved` turns it into a publish gate (Section 4.5).

Status: implementation-backed, with one boundary that was learned the expensive way.

Transition 13 exists because the guarantee above was once violated by the evidence record itself. Evidence is assembled from a dozen sources and written to a JSON column as the last act of a derive; a spreadsheet cell read as a native date-time value is not JSON-encodable, and the exception it raised inside the database write destroyed a completed paid run, leaving not even a broken draft. The repair is deliberately narrow: `derivation_service._persistable_evidence` attempts the encode and returns the evidence byte-for-byte unchanged when it succeeds, converting only on failure and recording in the evidence itself that a conversion happened. An unconditional stringify would have hidden the underlying defect, which is that something is putting live objects into a record. The honest limit is that this guarantees persistence against encode failures that have been *observed*; it does not prove the evidence assembly has no other way to fail.

4.2 Structural Gates

The structural gate is the question “does this specification run?”, asked of the deterministic layer rather than of the model that wrote it. Before this gate existed, derivation persisted whatever the agent returned; a specification declaring a lookup transform with no table to look in was

stored as an ordinary draft and discovered much later by a human running an offline checker. The agent was not being dishonest. Nothing had asked.

`spec_check.check_spec` returns every deterministic defect visible without the drawing gateway, as reviewable items carrying a code, the JSON path or field name, and a detail string. It has four parts, in a deliberate order.

Schema and shape. The IR validator rejects malformed documents, fields, extents, repeat blocks, row sets, merges, and record declarations. The IR shape validator alone accounts for 65 distinct `extttSPEC_*` codes out of roughly 200 prefixed error codes in the feature, and those codes are the vocabulary that the repair loop and the reviewer both read.

Policy. Runtime human input is disabled at derivation time, not merely at execution time, with `SPEC_USER_INPUT_DISABLED`. A field whose value comes from a person at run time is a residually defined field: it lets the party being scored decide which values it is not responsible for. Rejecting it at derivation rather than at execution also matters for the reviewer’s diagnosis, because the execution-time symptom is “a runtime parameter is missing”, which names the symptom and not the cause.

Source binding. `SPEC_SOURCE_FIELD_UNPRODUCED` reports a field whose declared source key nothing in the specification produces. It runs after the schema check, because a malformed field has no trustworthy source declaration to inspect, and before the trial compile, because the trial compile is precisely what hides this defect.

Trial compile. The specification is compiled against the derivation’s own sample data twice: once with every declared runtime parameter supplied and once with only the required ones, which are the two branches an optional override takes. This is the important half. The worst real defect it caught was a formula referencing a name that nothing provided

— perfectly valid syntax, and invisible to any schema check.

The gate is also defended against itself. A compile that raises rather than returns an error is caught and reported as `SPEC_COMPILE_CRASHED` rather than allowed to propagate, for the reason given in Section 4.1: an exception here occurs after the agent has been paid and before anything is persisted, so a single malformed rule would discard an entire run. The behavior is pinned by `test_the_trial_compile_survives_an_exception_it_cannot_anticipate`.

4.2.1 What a Structural Gate Catches That an Instance Run Cannot

The argument for spending effort here is that these checks run before any instance is processed and before any evaluation budget is spent. One concrete case makes the argument sharper than the general claim does. Two repeat blocks may write into the same set of slots, each reading its own row set, so that slot n takes its values from the n th row of two different tables. That is an *implicit positional join*, and its correctness depends on two conditions holding simultaneously by coincidence: that both row sets are ordered identically, and that both contain the same members. Remove the second condition — give one source file a project number the register does not contain, without changing the number of files — and every value shifts by one row. Row-set resolution succeeds. Both independent declared-quantity gates pass, because no quantity changed. Every written cell is a well-formed integer. Nothing downstream can see it.

The gate added for this shape, `SPEC_REPEAT_SLOT_ALIGNMENT_UNVERIFIED`, refuses a pair of blocks that share a slot origin and group size while reading different, unrelated row sets. Two things about it are worth recording as method rather than as engineering. First, the failure it addresses was present in a specification that had already passed every other check. Second, its collateral damage was measured rather than assumed: replayed against all thirty-four previously derived specifications it fires on exactly two block pairs, both true positives, and

zero times on the five specifications of the other evaluation dataset.

Status: implementation-backed, with one known ordering defect. When the schema layer reports an error, `check_spec` returns before running the trial compile, and the source-binding check is likewise skipped. The short circuit has a reason — a structurally broken specification has no reliable source declarations, and a compile error stacked on a schema error only restates it in less specific language — but the cost is that one derivation reveals one layer of problems, and seeing the second layer costs another paid run. This is a real defect against the cost argument of RQ3 rather than against correctness; it is recorded as an open item, and the intended repair is to run the source-binding check over the fields that are structurally sound instead of skipping it for the whole specification.

A methodological rule this section is an instance of. Adding a gate changes what the gates in front of it can still observe, and no test turns red when an older protection becomes redundant. Two protections in this system were measured to have become vacuous in their own test suites exactly this way. The discipline adopted in response is that a new gate is accepted only after each older gate it sits in front of has been disabled in turn and its own test confirmed to fail. This is the negative-control requirement of Section 4.3 applied to guards instead of to measurements.

4.3 Two Complementary Oracles

The structural gate establishes that a specification runs. Whether it is *right* is a different and much harder question, and it is asked twice, by two oracles that see different things. Neither subsumes the other, and the argument that they are complementary is the central empirical claim of this chapter.

4.3.1 The Author-Time Expected-Text Oracle

A derivation dataset that supplies an expected output already states what each target reference should read, and the derivation's own sample data is what produced it. The comparison is therefore available at author time, before any instance is processed and at no model cost: compile the specification against the sample, and compare the planned text at each reference with the expected document's text at that reference. `spec_check._expected_text_warnings` implements this.

What makes it worth a section is the class of defect that nothing else sees. Consider a detail-line quantity that the rule document defines as units per carton *times* the number of cartons, written instead as a sum. The two operands are small integers; the sum is 43 where the product is 120, and the amount that follows from it is 550.40 where the correct figure is 1,536.00. Both results are entirely normal-looking numbers: right currency, right decimal places, right thousands separator. The formula is valid, so the structural gate passes it. Before this oracle existed, the first indication would have been an evaluation run, after the money was spent, and only if the scorer itself had no correlated bug. The example is pinned by a test rather than merely described.

Three exclusions are necessary, and under-excluding is the dangerous direction, because every exclusion missed becomes an accusation against a rule that is in fact correct. A field fed by a runtime parameter is excluded, because the trial compile supplies a placeholder rather than the real input. A two-cell row is excluded, because without current values it collapses into a single string. Anything computing from a field that had to be dropped as invalid is excluded, because it is computing from a value the complete specification would have supplied differently.

Two extensions of this oracle are results rather than details.

The key must include the document role. Under multi-document specifications a target reference identifies a cell in *one* file and is not unique across a specification, so the same reference string exists in both output documents. Keyed by reference alone, the oracle told the agent that the packing list’s field was writing the wrong value; the repair loop obligingly rewrote that field to read the invoice’s key; and the specification was persisted reporting `ok : True` while planning a letter-of-credit number into a shipping-port field. **The system’s own correctness check manufactured a silent failure.** This is the strongest single argument in this thesis for the position that a verification mechanism must be verified with the same suspicion as the thing it verifies. The oracle was not absent, and it was not obviously broken. It was half-keyed, and a half-keyed oracle is worse than no oracle, because it actively drives repairs in the wrong direction.

The detail table was structurally invisible until three separate blockers were removed.

The detail table is the largest and most computed part of a specification — seven columns across fourteen slots plus six totals, in the DOCX evaluation dataset — and it was the only part with no author-time semantic check at all. Two blockers were known in advance: the comparable-reference map read only document fields, and a repeat cell had no author-time value, because its value came from resolving a locator against a workbook that the trial compile does not have. Resolving extents into row sets at transition 9 removes the second. The third was not anticipated: every cell of a detail table is written by a repeat block, so no *field* ever claims it, so it never appears in the per-field evidence — it lands among the unexplained changes instead. With both known blockers fixed and the third still present, the measurement was that all 140 repeat cells had a real author-time value and the oracle could compare *zero* of them, because the expected side of the comparison was empty. The number of mechanisms that were individually correct while their composition measured nothing is the point; this is the same shape as the

vacuous-guard problem of Section 4.2.

4.3.2 The Case-Level Oracle

The second oracle compiles the specification against a regression case’s own inputs and compares the resulting write plan against what that case says each target must read, yielding four verdicts per reference: `match`, `MISMATCH`, `MISSING`, and `OVERWRITE`. `OVERWRITE` is not redundant with `MISMATCH`: it is the verdict for rewriting a cell the case says must retain its baseline text, which is the negative obligation that a purely value-oriented comparison cannot express. References the specification writes that the case does not model at all are reported separately as out-of-scope rather than as failures, so a specification is not penalized for a cell the annotation does not cover.

The two oracles are complementary because they fail independently. The author-time oracle sees only the single calibration instance the derivation was given, so a rule that is correct for that instance and wrong in general — a constant copied from the sample, an extent that happens to end where the sample’s data ends — passes it and is caught only by the case-level oracle. The case-level oracle in turn needs annotated cases, which cost human effort per instance, and it runs after the specification exists, so it cannot prevent a defect from being proposed; it can only refuse it afterwards. Their combination is what allows the expensive oracle to be spent on the defects the cheap one structurally cannot see.

4.3.3 Why Not an LLM Judge

The obvious alternative to both is to ask a model whether the produced document is correct. This thesis refuses it, and the reason is specific to the primary risk measure rather than to a general preference for determinism.

Silent failure is an output that is complete, well-formed, plausible, and wrong. The errors a generating model makes and the errors a judging model makes are drawn from correlated distributions: they share a training distribution, a tokenizer’s numeric habits, and — if the judge belongs to the same family — a prior over what a filled form “should” look like. A judge is therefore least reliable exactly where the measurement matters most, because the outputs it is most likely to accept are the plausible-and-wrong ones. A judge would also make the measurement irreproducible, which forecloses the run-to-run identity this thesis claims for execution. The same reasoning governs the evaluator of Chapter 5, which is deterministic for this reason and not merely for convenience.

Status: implementation-backed, with three boundaries that must be stated.

1. **The author-time oracle sees only references whose text differs between the baseline and the expected document.** The comparable map is assembled from per-field evidence and unexplained changes, and both collect changed cells only. The consequence is exact, and belongs in the limitations rather than in a footnote: a specification that writes a value into a cell the expected document leaves *blank* is structurally invisible to this oracle — which is precisely the negative-obligation shape that the detail tables of both evaluation datasets depend on. What currently prevents the failure is that a misaligned block will almost always also collide with some cell whose content did change, and that is an accident of layout rather than a property of the design. The intended repair is a separate warning for “the specification claims a target the expected document leaves empty”, not an enlargement of the comparison map, because pouring every blank cell of a template into evidence would both inflate it and lose the distinction between deliberately blank and out of scope.

2. **Expected artifacts and the ground truth used to score them share an origin.** Both are produced by a hand-written per-dataset generator from the rule document, which satisfies the red line that expected outputs must not be produced by the system under test. It does not make them independent of *each other*: a misread rule produces a consistently wrong pair, and the manual recomputation that would catch it has been completed for one calibration instance of the XLSX dataset only. This is disclosed as a threat rather than repaired, and the direction of the resulting bias is toward agreement, not toward detecting error.
3. **Any measurement of zero requires a negative control.** A reported “no mismatches” is not evidence unless a deliberate corruption has been shown to make the count nonzero. This is not a stylistic preference: verification probes in this project returned a serene zero while measuring nothing on four separate occasions — from a wrong key name, a wrong field name, a comparison performed over an empty set after an upstream failure, and a control that changed every addition including a string concatenation, so that row-set resolution failed entirely and the result was 0/0 rather than a disagreement. A control must also distinguish doing the work from not doing it: an ordering test whose expected order coincided with the arrival order stayed green with the sort disabled, and had therefore measured nothing since the day it was written.

4.4 Case-Repair Loop

Both oracles produce failures that a model could plausibly correct. Allowing it to is a research risk and not merely an engineering one: the agent is being shown the expected answers, and “make case 3 pass” is trivially satisfied by a rule that special-cases case 3’s numbers and destroys case 2. Such a specification is *worse* than the broken one it replaced, because it now

passes the tests that were supposed to catch it. The repair mechanism is therefore defined by its refusals.

4.4.1 Two Loops, Deliberately Distinct

The derivation-time loop, `DerivationService._verified_spec`, runs inside transition 12 and closes over the structural gate and the author-time oracle. The case-level loop, `case_check.repair_against_cases`, closes over the case-level oracle. They share their anti-overfitting rules and differ in what a regression means, because the case-level loop has a per-case signal and the derivation-time loop does not.

4.4.2 Four Invariants

I1 — a repair may only touch the value layer. The keys an agent proposal may change are enumerated in one place: which formula, which lookup table, which conditional cases, which format, which constant, which source key, which aggregate. Target references, source locators, runtime parameters, and the set of fields are re-imposed from the original afterwards, so a “repair” can never quietly become a rewrite of where a value comes from or where it goes. Anything the agent attempted outside that set is reported rather than applied, which keeps a rejected proposal auditable instead of invisible.

I2 — a deterministic check owns the verdict; the agent only proposes. Every candidate is re-checked in full before it is taken, and a candidate that increases the error count is rejected outright: a repair that breaks more than it fixes is not a repair. Errors dominate warnings in that comparison, because trading a specification that compiles for one that agrees with the expected document but does not run is never the better deal. A candidate that ties on both is nonetheless accepted, since the agent may have rewritten a rule into a clearer form the checks cannot

distinguish.

I3 — at the case level, regression is judged by *which* references fail, not how many. A count is not enough: swapping a failure on one reference for a failure on another scores identically, so a repair could move the damage elsewhere and still be accepted as long as some other case improved. The guard therefore compares failure sets, and a case that already failed to compile is exempt, because otherwise the moment it started compiling would be scored as a regression.

I4 — the loop is bounded and hands off to a human. The derivation-time loop makes at most three attempts and stops early when the agent proposes nothing admissible, since an identical second round would only cost another call. The case-level loop stops after two consecutive rounds without a net gain. The rationale for stopping rather than continuing is substantive: at that point the ambiguous thing is usually the rule document itself, and no amount of additional model effort resolves an ambiguity in a requirement.

4.4.3 A Refusal That Was Correct

One incident is worth recording because it inverts the usual reading of a failed repair. Across two rounds the agent declined to add a source aggregate that the reported errors clearly called for. The diagnosis was not agent laziness: the repair prompt states that only the value layer may be modified, that key was not on the permitted list, and the resolved row set was not included in the repair payload. **The agent's refusal was the correct behavior for the contract it had been given.** The defect was in the contract. This is why invariant I1's permitted set and the repair prompt are treated as a single artifact: a repair loop constrained by two documents that disagree will spend paid calls producing correct refusals, and the symptom is indistinguishable

from a model that cannot do the task.

Status: mixed, and the mixture must not be blurred. The derivation-time loop is implementation-backed and runs on every derive; the four invariants above are pinned by tests, including tests that a repair may not move a target reference, may not invent or drop fields, and may not be accepted when it worsens any single case. The case-level loop is implemented and tested as pure functions, and is reachable through an offline operator tool that runs a stored specification against its cases without the drawing gateway; **it has no caller in the automated derivation path or in the API.** Following this thesis's rule for a gap between claim and implementation, the explicit route is taken rather than the silent one: the case-level repair loop is presented as a designed and tested mechanism whose automated integration is future work, its absence is recorded as an open item, and Chapter 5 must not attribute any measured result to it.

4.5 Human in the Loop: Evidence Review

The economic claim behind compile-once is not that human effort disappears but that it changes units: from reviewing every produced document to reviewing one specification once. That claim is only honest if the review is actually possible, which imposes requirements both on what derivation records and on what publication refuses.

4.5.1 What Is Recorded

A derived specification persists an evidence record with fourteen top-level keys. On the most recent derivation of the XLSX evaluation dataset they include: the agent's own analysis (its unresolved items with causes, its field inventory, its reconciliations, its rule dispositions, and the observed tool-call health), the per-reference changed text and its count, per-field confidence,

field renames, locator verification, row-set resolution, rule coverage, the specification-validation report, unexplained changes, unresolved labels, residual source keys, and version drift against the previous specification of the same lineage. Two of these exist specifically to keep movement from reading as churn: drift reports added, removed, and attribute-level changes, and the rename report identifies fields that are the same field under a new name. This matters because field names drift between derivations — measured twice — so field identity for review purposes is established by target reference, never by name.

4.5.2 What Publication Refuses

`gates.canonical_unresolved` merges every unresolved signal into one set, deduplicated by kind and reference: agent could-not-resolve items, unexplained changes in the expected document, unresolved labels, source locators that failed to resolve or disagreed with the sample, rule dispositions that are not covered, coverage-validation failures, and the specification-validation errors and warnings the repair loop could not clear. Any nonempty result blocks a normal publish. The design intent is stated in the module itself: nothing may be silently dropped into “ready to use”.

Publication is gated on that set together with template-fingerprint staleness, regression results, and human review. A human may override the reviewable gates, but the override is not free: it requires a written reason, and exactly what was bypassed is recorded permanently on the published version. An empty or structurally invalid specification is not overridable at all.

Two design decisions inside this gate are worth stating, because both were mistakes first. *Causes are carried, not collapsed.* An unresolved field says nothing on its own about why it is unresolved, so a gateway outage and a misunderstood rule appear identical in the specification while being fixed in completely different places. The run’s observed tool health is therefore

folded into each unresolved item's cause, and the review interface says so first and plainly, because a reviewer who read an outage as a coverage regression went looking for a rule that was never missing. *The headline number is covered, never accounted.* Accounted is the total minus the unmentioned, so it counts a rule the agent enumerated and then admitted it could not implement as though the work were done: fifteen rules implemented, sixteen reported unresolved, and zero unmentioned, once rendered as a green “34/34”. Both halves of that display were wrong in the same direction — the flattering one — which is why the direction of an error, and not only its size, is treated as a reportable property throughout this thesis.

Status: implementation-backed as a mechanism; normative as an economic claim. The evidence record, the merged unresolved set, the forced-publish audit trail, and the review interface all exist and are exercised by tests. What is *not* measured is the quantity the economic argument needs: how long a review takes, how much of the evidence a reviewer actually uses, and whether a reviewer detects a defect the gates missed. No user study has been conducted. Chapter 5 therefore reports human cost only as the count and kind of items a reviewer is required to adjudicate, and the threats-to-validity discussion discloses that this understates human cost, since adjudicating an item is not free.

4.6 Coverage Is the Real Bottleneck

Once a specification's fields are individually accurate, the residual risk is dominated not by fields that are computed wrongly but by rules that no field implements at all. A rule the specification never addresses produces no field, no unresolved entry, and no gate: it is invisible by construction. Measuring that invisibility is therefore a methodological problem before it is an engineering one.

4.6.1 Why the First Denominator Did Not Work

The original implementation asked the agent to enumerate the rule document and counted its enumeration. This cannot answer the research question, for two independent reasons. A rule the agent never noticed is absent from the denominator, so “22 of 22 covered” and “22 of 40 covered” are the same observation; and omitting a rule *raises* the score, which points the incentive the wrong way. Worse, the visible figure was not comparable between runs: one derivation counted 39 mapped fields and the next 28, reading as a large regression, when nine of the 39 were filler the compiler now handles itself and the truth was a small improvement. Unioning inventories across derivations made omissions monotonic but left the evaluated agent choosing the items being counted.

4.6.2 The External Denominator

The active mechanism externalizes the denominator. A rule document must be accompanied by a frozen, versioned rule-applicability manifest that lists rule identifiers, their prose, and the document roles each applies to — and nothing else, in particular no target coordinates, which would make the manifest an answer key. The manifest is bound to the rule document by the SHA-256 of its exact bytes and to the dataset by its declared document roles, and a derive without it is refused before the agent runs.

The agent supplies a *disposition* for each externally assigned identifier and can do nothing else to the denominator: unknown identifiers are discarded, duplicate dispositions are refused, and an assignment whose document roles disagree with the manifest is refused. The resulting partition into `covered`, `unresolved`, `rejected`, and `unmentioned` is mutually exclusive and sums to the frozen applicable set. A claimed implementation is checked, not accepted: a `covered` disposition must name at least one field that really exists in the specification under

a role where the manifest says the rule applies, and otherwise becomes `unmentioned` plus a deterministic issue. Everything that is not `covered` blocks publication; there is no partial credit.

4.6.3 Metric Plumbing Is Methodology

The most consequential finding of this section is not the manifest. It is that a denominator, an allow-list, or an accepted spelling silently decides a reported number without failing anything, and that this failure mode is separate from — and harder to see than — a wrong mechanism. Three measured instances make the point.

1. **A denominator that admitted the wrong checks.** The scoring policy initially counted the “this slot must remain blank” obligations — 120 of them on one instance — inside the semantic denominator. An arm that submitted the blank template unchanged, writing nothing at all, therefore scored 66.7% on semantic correctness. After separating those obligations into their own channel it scores 0%. Every individual check was correct; what was wrong was which total they were added into.
2. **An accepted spelling that decided the headline.** Rule dispositions may name a field either bare or qualified by its owning block. One derivation used the qualified form throughout; the function computing the numerator accepted only bare names, so eleven correct assignments were reported as naming an unknown target, and coverage was displayed as 8 of 27 when the specification in fact covered 19. The specification and the assignments were both correct. A naming convention had decided the thesis’s headline diagnostic number.
3. **Declared data that nothing read.** Each range in the benchmark manifests declares, for every wrong-extent hypothesis, the values that hypothesis would produce. The shared

evaluator did not read that field at all. Once connected, an output that occupies one extra detail slot while leaving all six totals byte-identical passes all 112 semantic checks and is nonetheless *named* as a specific wrong-extent hypothesis. The detection capability had been authored, and was simply not wired up.

The rule adopted from these is stated as a verification obligation rather than as advice: whenever a headline figure moves, confirm that the mechanism changed and not the bookkeeping.

4.6.4 What Coverage Does and Does Not Measure

Coverage is a derivation diagnostic. It states which rules the specification claims to implement, verified against an external list; it does not state that an implemented rule is correct, which is the oracles' question, and it must not be substituted for evaluation correctness.

A single coverage number for a dataset is also an incomplete description of the method, and the honest form is a distribution. Four independent derivations of the same XLSX dataset — with the prompt, the payload, and the input bytes identical — produced coverage of 15, 12, 19, and 23 out of 27 respectively, alongside structural differences of the same magnitude: one run declared three detail slots where the template has seven, another declared none of the cross-file collection its predecessor had used, and the number of hard structural errors ranged from four to zero. This variance is not noise around the method's performance; it *is* the method's variance, and the formulation of Chapter 3 predicts exactly where it lives. Because $\mathcal{A}$ is stochastic while $\mathcal{X}$ is deterministic, all of the run-to-run variance of this approach is concentrated in derivation and none of it is in execution. That is the substance of RQ2, and reporting a single derivation's coverage without the distribution would misstate it.

Status: implementation-backed, with the residual gaps named. The external denominator, the four-state partition with its checksum, the pre-agent admission of the manifest, the numera-

tor validation, and the publish gate are all implemented and tested. Three things remain outside the mechanism. First, the manifest is authored by a human from the rule document; binding it to the document's bytes prevents drift but does not prove the human segmented the prose correctly, and no independent segmentation of the rule documents has been performed. Second, the applicable-set hash proves which rules were counted, not that the set is the right one. Third, coverage says nothing about a rule that is covered *wrongly*; that separation is deliberate, and it is why this chapter needs both this section and Section 4.3.

4.7 Boundaries of Author-Time Verification

Section 3.1.4 listed what the IR cannot express. This section lists what the derivation procedure cannot see, which is a different set. Each item is stated in the form that makes it checkable: what is claimed, what is not, and which direction the resulting bias runs.

D1 — an expected output supplies target coordinates that introspection alone does not.

When the dataset includes an expected document, the difference between baseline and expected locates the cells that must be written, including cells the baseline leaves blank. That is what makes blank-target binding work at derivation time. It does not follow that an unmarked blank paragraph or cell has a stable semantic anchor when only the template is available: without the expected document there is frequently nothing to distinguish one blank from its neighbours. Derivation-time capability and introspection-only capability are therefore separate claims, and only the first is demonstrated.

D2 — the DOCX paragraph-blank locator is a pilot-supported volatile locator. The mechanism that addresses a blank run inside a Word paragraph resolves against the literal textual context surrounding it in the template. It is supported for the DOCX evaluation dataset and is not

presented as general Word blank-target addressing; a template edit that changes the surrounding wording can invalidate it, which is why the write-back layer distinguishes stable content-control and bookmark identities from positional ones and fails loudly on the latter rather than guessing.

D3 — the label-proximity mechanism has never executed on either main evaluation carrier. Labeled-field resolution is constructed only on the non-agent derivation path, and both evaluation datasets derive through the agent. Of thirty-three stored specifications, the thirteen fields whose target was matched geometrically all belong to three DWG specifications from an earlier period; the DOCX and XLSX datasets contribute zero. This thesis therefore does not claim that labeled-field location works on DOCX or XLSX, and the measurement that recently tightened that mechanism was likewise taken entirely on DWG specifications and does not transfer.

D4 — the author-time oracle sees only references whose text *changed*, as established in Section 4.3. It compares text against text, so a target that is blank in the baseline and blank in the expected document produces no diff entry and is never compared. Two of the three failures that follow from this have since been closed, and the third has not; separating them matters, because the original single statement of D4 was used to justify a protection that turned out to be a coincidence.

The first failure — the specification writes a value into a reference the expected artifact keeps blank — is now detected by a distinct check rather than by the oracle. Derivation records the set of references at which each expected artifact holds text, and any planned non-empty value at a reference outside that set is reported. On a fixture whose repeat block is offset by one row, it names the two cells carrying a project name in a row required to stay blank; the test that pins that fixture previously recorded, in words, that the oracle said nothing about them. Two

silences in that check are load-bearing and are kept: a planned blank never warns, because an unused slot resolving to nothing is required behaviour and warning on it would accuse every specification whose calibration instance is shorter than the template; and a document for which nothing was recorded never warns, because absent evidence is not evidence of a blank.

The second failure — the specification cannot state that it deliberately writes nowhere — is now expressible, in two forms. Where a repeat block declares column positions for only some of its record's row roles, the uncovered positions are *derived* as implied blanks and written blank, with no new key in the representation: on the XLSX dataset this yields 21 cells that the block owns and blanks. Where the cells lie outside every block — a signature box, for instance — the document declares them, and the declaration is checked at derivation time against three conditions: every reference must be addressable on the template, must be blank in the expected artifact, and must not be written by anything else in the specification. A declaration failing any of the three is refused and does not reach the persisted specification, which keeps it out of the coverage numerator; this construct is the only one in the representation that *relaxes* a publish gate, and without those conditions an agent could dispose of any rule it failed to locate by swearing the cell stays blank.

The third failure remains open and is the residual meaning of D4: a reference that neither the expected artifact nor the specification writes, and that no rule names, is invisible. There is nothing at it to compare and no declaration pointing at it. The bias direction is unchanged and is towards accepting a specification: the oracle can only accuse, and a reference it cannot see is never accused.

D5 — an unpublishable specification is not by itself evidence of a failed derivation, and the count must be decomposed before it is read. Two separate findings sit under this heading, and both were closed as defects rather than being boundaries.

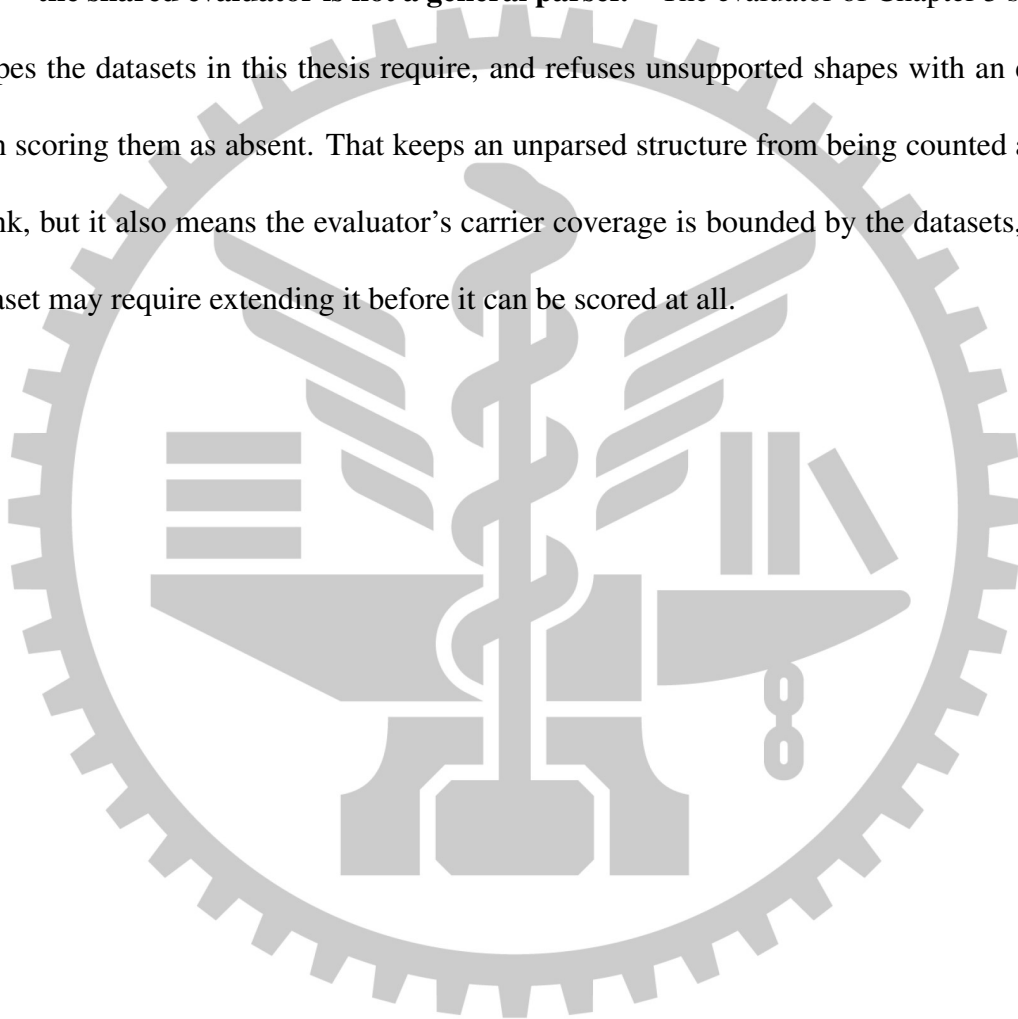
The first is that the unresolved set was computed as “every field without a verified target”, which does not distinguish an output field whose target could not be located from an *intermediate* field that legitimately has no target — one carrying a located source value into another field’s formula and never written anywhere. The most recent XLSX derivation contained five such fields, each with a working source locator, each read by a header-summary formula, and each recorded as blocking publication. The correction has two halves and the order between them is not optional: intermediate inputs were first admitted to the executable locator set, and only then were they exempted from the unresolved report. The exemption is safe solely because a new hard error now refuses any specification whose formulas read a name that nothing executable can supply. That error was not decoration: replayed across the thirty-five stored specifications it named eleven, and each of the eleven genuinely fails to compile, with no false positive. Before it existed, the two specifications this thesis had recorded as its best both stored a successful validation result and both failed at execution with an unknown formula field.

The second is that two counters reported the same quantity and were never compared. Every narrative of “ n unresolved items” in this project’s records is the length of the agent’s own self-reported list; the publish gate counts a different, canonical set. On the best XLSX specification the two read 13 and 47, and 34 of the 47 were detail cells that the repeat block writes correctly — admitted to the gate’s input because the author-time oracle reads expected text from that same list, for a different question. The gate now excludes cells a block owns, recomputed from the specification rather than read from an evidence tag, so that a later version which removes the block puts the same cells back to blocking. No test failed while this was wrong, because what was wrong was a count.

What remains true, and is why D5 stays in this list: after both corrections the XLSX specification is still unpublishable, and decomposing the residue shows the gate is right to block

it. One of the remaining rules requires a value in a column that the specification declares no field for at all, and execution confirms the consequence — three cells scored as silent failures on instances the derivation never saw, because the calibration instance happened to leave that column empty. A count of unresolved items conflates a gate that is too strict with a gate that is doing its job, and the two are repaired in opposite directions.

D6 — the shared evaluator is not a general parser. The evaluator of Chapter 5 supports the shapes the datasets in this thesis require, and refuses unsupported shapes with an error rather than scoring them as absent. That keeps an unparsed structure from being counted as a correct blank, but it also means the evaluator’s carrier coverage is bounded by the datasets, and a new dataset may require extending it before it can be scored at all.



Chapter 5. Performance Evaluation

5.1 Research Questions

The claim under test is not that a language model can fill in a form. It is that, for a bounded class of recurring template-instantiation tasks, the model can be removed from the recurring execution path altogether without losing correctness, traceability, or fail-loud behaviour. Four research questions decompose that claim; a fifth asks which components of the derivation procedure carry it.

RQ1 — Correctness. Is a compiled specification’s field-level semantic accuracy at least as good as a per-instance agent’s on the same instances, measured by the same deterministic evaluator?

RQ2 — Consistency. How much does repeated execution of the same task vary? The question is asymmetric by construction, and the asymmetry is the point: for a per-instance agent the variance is present in every one of the N executions, whereas for the compiled method execution is deterministic and *all* of the variance sits in the single derivation. RQ2 therefore has two halves, and reporting only the first would be a misstatement: run-to-run variation of the baselines, and derivation-to-derivation variation of the compiled specifications.

RQ3 — Cost structure. What is the shape of cost against instance count? A one-off derivation cost plus a near-zero marginal execution cost crosses a per-instance cost at some N ; the

question is where that crossing lies, and how sensitive it is to derivation retries.

RQ4 — Behaviour under template drift. When the template changes underneath a frozen artifact, does the method fail loudly and locate the change, and do the baselines fail silently?

This is the direct test of the design principle that governs the whole system.

RQ5 — Component contribution. Which of the structural gates, the two oracles, and the repair loop are load-bearing, and which are redundant with each other?

An earlier version of this plan carried a sixth question — whether the method generalizes beyond drawings to office documents. It has been removed rather than answered, because DOCX and XLSX are now the primary evaluation carriers rather than a proof of concept, and a question whose answer is a premise of the study design is not a research question.

A revision to the emphasis, and the measurement that forced it. The original framing put RQ1 first and treated RQ2 and RQ3 as secondary. A pilot measurement on the DOCX dataset changed that ordering. Running a per-instance agent three times over one calibration instance produced semantically correct output every time — the cross-file lookup, the rounding, the gram conversion, and the extent decision were all right — and three different files: three distinct output hashes in three runs, and cell-level agreement of 119 of 172 cells, or 69.2%. All 53 divergent cells were formatting decisions retaken from scratch on each run, a thousands separator present in one run and absent in the next, and none of them was a semantic error.

The consequence is stated here rather than in the results, because it is a claim about what this thesis argues: the primary contribution is not that the compiled method is *more accurate* than a competent agent on tasks a competent agent can do. It is that accuracy is comparable while consistency and cost structure differ structurally. Two tempting responses to that pilot were considered and rejected. Raising task difficulty until the baseline fails would tune the

benchmark towards making the comparator fail; substituting a weaker model would be choosing the opponent. Both are validity threats, and both are recorded as rejected in Section 5.10.

Status: implementation-backed as a measurement, normative as a research plan. The pilot above was executed and its artifacts are retained. The five questions are a plan; the formal experiment has not been run, and no figure in this chapter should be read as an answer to any of them.

5.2 Study Design

5.2.1 Why Not a Few-Shot Held-Out Split

The conventional evaluation for a language-model system is a few-shot held-out split: the model sees K demonstrations and is scored on unseen tasks. That protocol borrows its meaning from in-context learning, and this method is not that shape.

The template is an *input*, not the object of generalization. The system is given the template on purpose; so is every baseline, on every one of its executions. Seeing the template is therefore not leakage but a premise, and it is symmetric across arms. This makes the standard split either vacuous or unfair depending on how it is drawn. If the held-out instances share a template with the demonstrations, nothing was held out that matters: the compiled specification and the agent’s demonstrations both describe that template. If the held-out instances come from a different template, the compiled method must derive again — the demonstrations do it no good at all — and the comparison measures the cost of a derivation rather than generalization from examples.

What the split is genuinely needed for is narrower, and is retained: preventing the situation in which one arm’s prompts, gates, or intermediate representation were tuned on the very data it

is scored on while the baselines were not. The instrument for that is separation at the *template* level, and Section 5.2.2 reports how far that separation actually holds in this study.

Status: normative, and deliberately so. This subsection argues a design choice. It is included because the objection it answers is the first one a reader of a language-model-systems paper will raise, and because declining a standard protocol without stating why reads as avoiding it.

5.2.2 Two-Level Split

The split has two levels, and only the first of them currently holds in full.

Instance level. Every evaluation template carries ten instances: $K = 3$ calibration instances, shared by all three arms and the only instances any arm may see during construction; two development instances used for debugging; and $M = 5$ sealed evaluation instances that no arm sees at any stage. Two of the five evaluation instances in each dataset are cases whose *correct* behaviour is a refusal, so that an arm which always produces a file cannot score well merely by producing files.

Template level. The evaluation templates are two, with a third planned:

- **S4 (DOCX):** a Commercial Invoice and a Packing List produced together from a product master file and a shipment detail file. Its principal difficulty is cross-file computation — unit price from one workbook, quantity from another — and it is the only DOCX evaluation dataset, so every DOCX claim in this thesis rests on it. Thirty-four applicable rules.
- **X1 (XLSX):** the Directorate General of Highways monthly site-audit report chain, in

which N real 13020A audit sheets are aggregated into one 13020D monthly report. Its principal difficulties are a cross-file collection of source artifacts and a layout in which one logical record occupies two physical rows. Twenty-seven applicable rules.

- **S5** (XLSX, not yet built): a credit-programme completion audit, chosen for its density of negative obligations.

Two further items must be named, because their absence changes how the split reads. **S3** is a synthetic pilot dataset that was built to protocol and then rejected on the grounds that its layout and content were not those of a document that really circulates; it is retained only as a pilot for the evaluator and the range locator, is not in the main results table, and is not offered as evidence of external validity. **S1** and **S2** appear in the protocol document as a proposed assignment of two development templates and *were never built*.

The gap this leaves, and the direction of its bias. Because **S1** and **S2** do not exist, development did not happen on templates disjoint from the evaluation set. It happened on the DWG case study, on the rejected pilot, and — decisively — on the calibration instances of **S4** and **X1** themselves. Three intermediate-representation primitives were added specifically so that the compiled arm could produce **X1** at all: artifact collections, stacked records with a slot origin, and equality merge between resolved row sets. **X1** is an evaluation template. The template-level separation the protocol describes is therefore, at present, a normative claim rather than a property of this study, and its bias runs *in favour of the proposed method*.

Following the third of the three admissible responses to such a gap, the claim is kept and labelled rather than quietly weakened, and the bias is disclosed again in Section 5.10. Two measurements bound its size without cancelling it. First, each primitive was added only after it had been observed to fail on a real file, and its collateral effect on the already-stored specifications

was swept: the slot-alignment gate introduced with the third primitive was replayed against all thirty-four stored specifications and matched exactly the two it was written for, with no false positive on the five specifications of the other dataset. Second, value-level verification of the resulting specification was performed on development instances the derivation never saw: the fourth X1 specification reproduced 255 of 255 scored values across the three calibration and two development instances using the scoring policy’s own comparator, and a deliberately corrupted variant of the same specification scored 240. Neither measurement makes the development location correct; they bound how far it could have been exploited.

A second construct-validity item, recorded here and again in Section 5.10. X1’s rules require projects to be reported in ascending project-number order. The real 13020D form prescribes no ordering; the ordering is a convention this dataset introduced so that the extent has a checkable manifestation. The mechanism that implements it is built and tested, but the question of *who decided the order* belongs to dataset construct validity rather than to the method.

Status: implementation-backed at the instance level, normative at the template level, with the bias direction stated.

5.2.3 Dataset Construction

5.3 Baselines and Fairness

Three arms are compared. They differ only in what persists between instances.

Arm A — per-instance agent. For each instance, an agent is given the template, the rule document, and the source data, and produces the output document. Nothing persists: the next instance starts from the same prompt with no artifact carried over. This is the status quo

the thesis argues against, and it is deliberately a strong version of it — a current frontier model with the same structured inspection tools the proposed method uses.

Arm B — agent-built frozen skill. The agent is first allowed to generalize from the calibration instances into a reusable artifact of its own design — a script, a checklist, a set of instructions — which is then frozen. Every evaluation instance is executed by an agent that may read that artifact and may not modify it.

Arm C — the proposed method. One derivation produces a specification; every instance is executed by the deterministic compiler with no model call at all.

Why three arms and not two. Arm B exists to answer the strongest available objection to the result. If only A and C were compared, any advantage of C could be attributed to the mere existence of a persistent artifact rather than to anything about how that artifact is produced and verified. Arm B holds persistence fixed and varies only the production procedure: an agent generalizing freely, versus a derivation constrained by an intermediate representation, structural gates, and two oracles. The question Arm B answers is therefore the one that actually distinguishes the contribution — is the verified derivation better than an agent’s own attempt at the same idea?

Fairness has two layers, and the second is the one usually omitted. The first layer is *tooling parity*: the same structured dump utilities, the same model, the same retry ceiling, the same wall-clock ceiling. The second is *material parity*: Arms A and B receive an input list that is byte-identical to the one Arm C’s derivation receives, because an advantage produced by handing one arm a cleaner or richer set of inputs is not an advantage of the method. The frozen input manifest is truth-free by construction and its envelope is hashable, so what each arm was given is recorded rather than asserted.

Material parity has a specific consequence that must be stated because it works against the proposed method. The rule material given to any arm may not contain target coordinates — no cell addresses, no bookmarks, no content-control tags, no drawing handles. Arm C's derivation therefore has to locate its own write targets, and cannot be given them; the rule table is not permitted to become an answer key for anyone.

Arm B's freeze must be enforced by the filesystem. An instruction not to modify the skill is not a control. The frozen artifact is hashed and mounted read-only for the evaluation phase; an attempt to write to it is recorded as a protocol violation and reported rather than silently tolerated. This matters because the failure it prevents — an agent quietly improving its own skill between instances — would convert Arm B into a per-instance arm while still being reported as a frozen one.

Status: contract implementation-backed, live execution not yet built. The three-arm input and evidence contract is frozen and tested: the truth-free input manifest, the hashable envelope, the Arm B freeze, namespace isolation for Arm C, retention of raw attempt evidence, and fake-capture negative controls are all implemented. The live adapters that actually drive Arms A and B, and the formal wrapper around Arm C, are not. The pilot reported in Section 5.1 was run outside that harness and is labelled as a pilot for exactly that reason; it must not be read as an Arm A result.

5.4 Metrics

5.4.1 Asymmetric Rerun Budget

5.5 The Evaluator

All three arms are scored by one deterministic program. It parses the produced artifact, extracts the values at the locations the dataset’s annotation manifest names, compares them against that instance’s ground truth, and emits per-check results. It never asks a language model whether an output looks right, for the reason given in Section 4.3: a judge that can be wrong in the same direction as the system under test cannot bound the system’s error.

5.5.1 Separate Channels, Never One Average

Scores are partitioned into channels, each with its own denominator: *semantic* (the primary metric), *extent* (which source rows were included), *refusal* (a case whose correct behaviour is to refuse), *structure* (the artifact opens and has the expected shape), *presentation* (formatting, reported separately from meaning), *untouched* (regions no field declares were not modified), and *integrity* (the package’s critical parts survived). A check whose prefix is not one of the declared channels fails closed rather than being scored as absent. The policy is versioned, and every result record carries the evaluator version, the policy version, the source hash of the scoring code, and the repository commit.

The partition is not presentational. The one measurement that settles it: while the “this slot must remain blank” obligations were counted inside the semantic denominator, an arm that submitted the blank template unchanged — writing nothing whatsoever — scored 66.7% semantic correctness on one instance. After the obligations were moved into their own channel

the same output scores 0%. Every individual check had been correct; what was wrong was which total they were added into.

5.5.2 Two Self-Validations, Neither Sufficient Alone

Forward. The evaluator must score the generator’s own expected artifacts at 100%. This catches an evaluator that reads the wrong location or compares with the wrong tolerance.

Reverse. Controlled corruptions are applied to the expected artifact and the evaluator must catch every one. Without this, an evaluator that always answers “pass” would give every arm a perfect score and the forward test would not notice. The current mutation coverage is 6 of 6 on the pilot dataset, 12 of 12 on S4, and 13 of 13 on X1; X1 is the first dataset in which *all seven* channels have at least one mutation that exercises them, which closed two channels that had previously been structurally vacuous.

A mutation test is not a validity proof, and the gap is specific. The expected artifact and the ground truth are produced by one generator. If the generator computes a value wrongly, both are wrong together and the evaluator faithfully reports 100%. Mutation testing establishes that the evaluator detects corruption; it says nothing about whether the generator computed correctly. The two do not overlap. The present detection mechanism for that residual is a per-dataset spot audit — at least three fields per dataset recomputed by hand from the rule document, including one aggregate and one negative obligation — which is probabilistic rather than a guarantee, and is reported as such.

5.5.3 A Negative Control for Every Zero

The evaluator and the probes around it are themselves software, and the failure mode observed repeatedly during construction is that a broken probe reports a serene zero: zero mismatches, zero problems, full marks. Measured instances during this work include a probe that used the wrong key name, one that omitted a worksheet prefix from every reference so that nothing matched anything, one whose trial compilation failed so that the comparison returned an empty list, and one that compared string forms where the annotation declared exact decimal comparison, so that twenty *correct* cells were reported as wrong. Two of those report “looks broken” and the rest report “looks fine”; the second kind is the dangerous one, and it is indistinguishable from success by inspection.

The rule adopted is therefore mechanical rather than advisory. Any reported zero or perfect score must be accompanied by a deliberate control that makes the figure move, and the control must distinguish doing the work from not doing it — a control whose expected value also happens to be the result of skipping the mechanism measures nothing and never turns red. That last clause is not hypothetical: an ordering test was found whose expected row order equalled the arrival order, so disabling the sort entirely left it green.

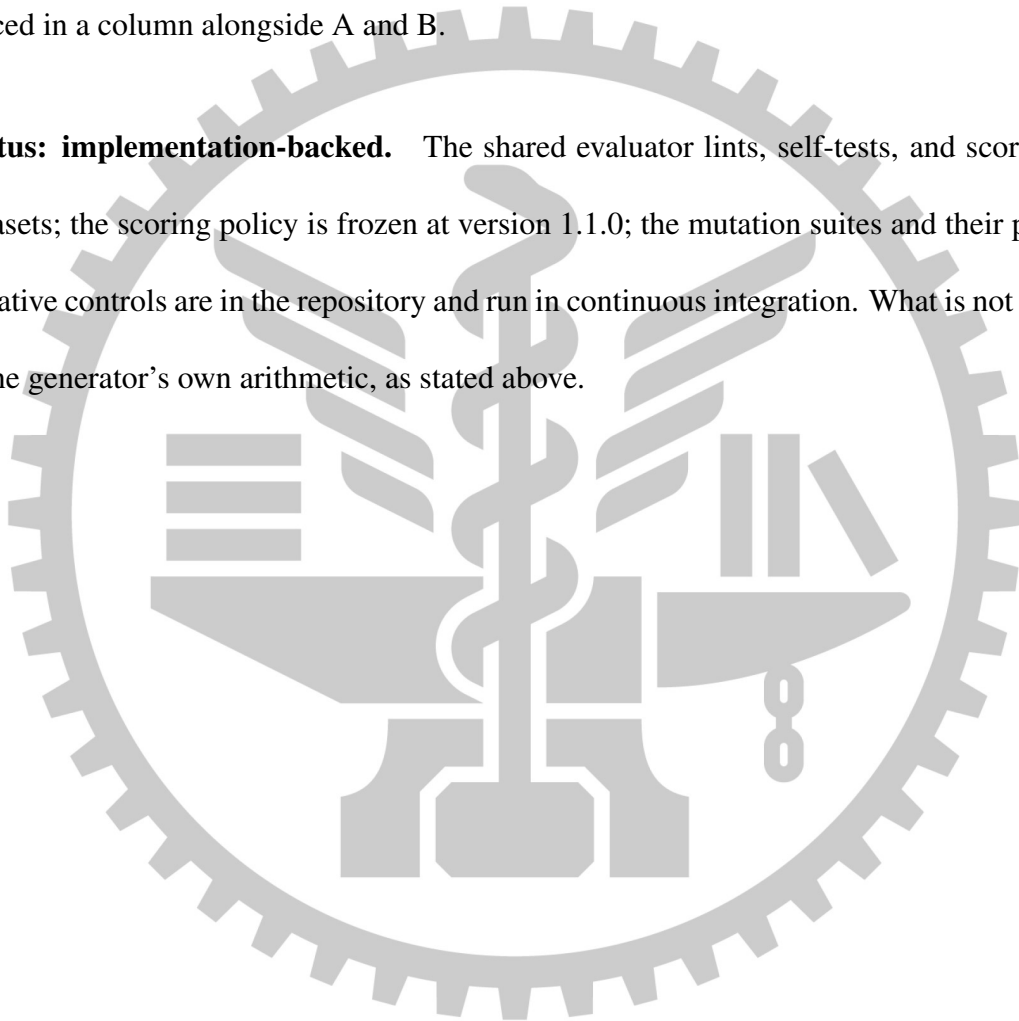
5.5.4 The Extent Diagnostic Is Not Symmetric Across Arms

Only Arm C declares which source rows it included; Arms A and B produce a document and no provenance. Scoring the extent channel from an arm’s own declaration would repeat exactly the mistake that the coverage denominator made in Section 4.6 — a figure self-reported by the system being scored is not a cross-arm metric.

The cross-arm instrument is therefore a set of *cross-checks*: identities between two independently scored fields of the same instance, which any arm’s output either satisfies or does

not. Alongside them, each dataset's manifest declares, for every wrong-extent hypothesis, the values that hypothesis would produce, so a wrong extent can be *named* from the output alone rather than inferred. The effect is measurable: an output that occupies one extra detail slot while leaving all six totals byte-identical passes all 112 semantic checks on one instance and is still identified as a specific wrong-extent hypothesis. Arm C's own resolved anchors are reported as supplementary evidence and are explicitly marked as available for one arm only; they are never placed in a column alongside A and B.

Status: implementation-backed. The shared evaluator lints, self-tests, and scores all three datasets; the scoring policy is frozen at version 1.1.0; the mutation suites and their per-channel negative controls are in the repository and run in continuous integration. What is not established is the generator's own arithmetic, as stated above.



5.6 Template Drift

5.7 Ablations

5.8 Results

5.8.1 Where the Randomness Went

5.9 Case Study: Engineering Drawings

5.10 Threats to Validity

Each item below states what is threatened, what was measured, and which arm the bias favours. Items whose direction favours the proposed method are listed first, because those are the ones a reader has no independent way to discover.

5.10.1 Threats Biased Towards the Proposed Method

T1 — development happened on evaluation templates. The template-level separation described in Section 5.2.2 does not hold: the two development templates the protocol proposes were never built, and the prompts, gates, and intermediate representation were tuned against the calibration instances of the evaluation templates themselves. Three representation primitives exist specifically because the compiled arm could not otherwise produce X1. The bias favours Arm C, and it is not quantified — the only way to quantify it is to build the missing development templates. The two bounding measurements are given in Section 5.2.2.

T2 — the cell-datatype exemption. The compiler stringifies every value before writing it, so a numeric field lands in a spreadsheet as text. The annotation rules therefore declare `presentation.dat` as `any` for every field. Without that exemption Arm C would lose a point on every numeric field, and the study would be measuring a write-side type policy rather than the method. With it, a real defect — numbers stored as text — is not scored, and a baseline that writes correct types earns nothing for doing so. This is the third admissible response to a gap: the claim is kept, labelled normative, and the direction disclosed. It favours Arm C.

T3 — the rule tables are ours. Even where the template is a genuine third-party artifact, the machine-readable rule applicability manifest that supplies the coverage denominator was written for this study. It weakens the objection that the templates are self-designed; it does not weaken the objection that the rules were written by someone who knew what the method can express. Any use of an externally sourced template must state this in the same breath, or it reads as padding.

T4 — three primitives rest on a single example. Artifact collections, stacked records, and equality merge between row sets were each validated on one dataset, and it is the same dataset in all three cases. The key–value extent added for the DOCX dataset has the same property: a survey of six real government forms found no second instance of the shape. The failure mode this predicts is not a red test but a different form that silently does not fit, and no measurement in this thesis would detect it.

5.10.2 Threats Biased Against the Proposed Method

T5 — package preservation was, until recently, unmeasurable for Arm C. The integrity channel treats drawings and printer settings as critical parts of a spreadsheet package, which

is why the government form was chosen as a dataset in the first place. Until it was corrected, the production write-back for XLSX serialized the workbook through a spreadsheet model, and that operation drops exactly those two parts — the same two parts the mutation suite’s integrity negative control drops, because the negative control *is* a round trip through the same library. Arm C therefore scored 0 of 5 on integrity by construction, and the channel could not distinguish a correct run from a deliberately corrupted one.

The response was the first admissible one: the write-back was replaced by a package editor that edits the one worksheet part that must change and copies every other byte through. The measurement after that change, on the same five instances, is 5 of 5 with no preservation violation, while re-serializing the same outputs through the spreadsheet model still yields 0 of 5. Narrowing the channel to fit the writer was considered and rejected, because it would have been a reduction of the thesis claim to match the implementation. The threat is recorded rather than deleted, because every integrity figure produced before that correction is an artifact of the writer and not a result.

T6 — the DOCX write path has not had the same audit. The correction in T5 covers XLSX only. The layout-bearing parts of a Word package have not been enumerated part by part, and S4 is the only DOCX evaluation dataset. Any DOCX integrity figure in this thesis therefore carries an unmeasured residual.

T7 — an unpublishable specification is not necessarily a failed one. The publish gate blocks on any unresolved item, and the unresolved set has repeatedly been found to contain entries that are not defects. Two counters existed for the same quantity — the agent’s own self-reported list and the gate’s canonical set — and they were never compared: on the best X1 specification they read 13 and 47. Thirty-four of the 47 were detail cells that a repeat block

writes correctly, admitted to the gate’s input list because the author-time oracle reads expected text from that same list for a different purpose. Any count of unresolved items must be decomposed before it is interpreted, and a headline “unpublishable” figure without that decomposition would understate the method.

5.10.3 Threats of Undetermined or Mixed Direction

T8 — dataset scale. Two evaluation templates exist and a third is planned, against an original intention of three to five per carrier. Conclusions must be descriptive; no statistical significance may be claimed, and no single average may be taken across carriers.

T9 — the inclusion criterion is itself skewed. The datasets admit only tasks whose target row count is fixed, because the write side deliberately does not insert rows, and they exclude formula results. Both exclusions coincide with boundaries of the proposed method, so the excluded region may contain tasks the baselines can do and the compiled method cannot. This is the second admissible response — an explicit expressiveness boundary — and it is disclosed rather than argued away.

T10 — what the untouched channel does not reach. Untouched-region comparison covers cells and paragraphs. It does not reach footnotes, endnotes, comments, or other independent package parts, and styles and formula results are outside the first version of the scoring policy by design, to avoid confounding correctness with whether a spreadsheet application refreshed its cached values.

T11 — model dependence. Every derivation reported here used one model family. The ablation in Section 5.7 is the instrument for this and has not been run; until it is, no claim in this thesis is established as vendor-independent.

T12 — the correctness of the reference outputs. Discussed in Section 5.5.2. Mutation testing bounds the evaluator, not the generator; the residual is covered only by a probabilistic spot audit.

T13 — two responses to the pilot that were rejected, and why. After the pilot in Section 5.1 showed the per-instance baseline to be semantically accurate, two adjustments were available and both were declined. Increasing task difficulty until the baseline begins to fail would tune the benchmark towards producing the desired comparison, and substituting a weaker model would amount to selecting the opponent. Recording them here is part of the claim: the emphasis of this thesis moved to consistency and cost structure *because* the baseline was accurate, not because the tasks were made harder.

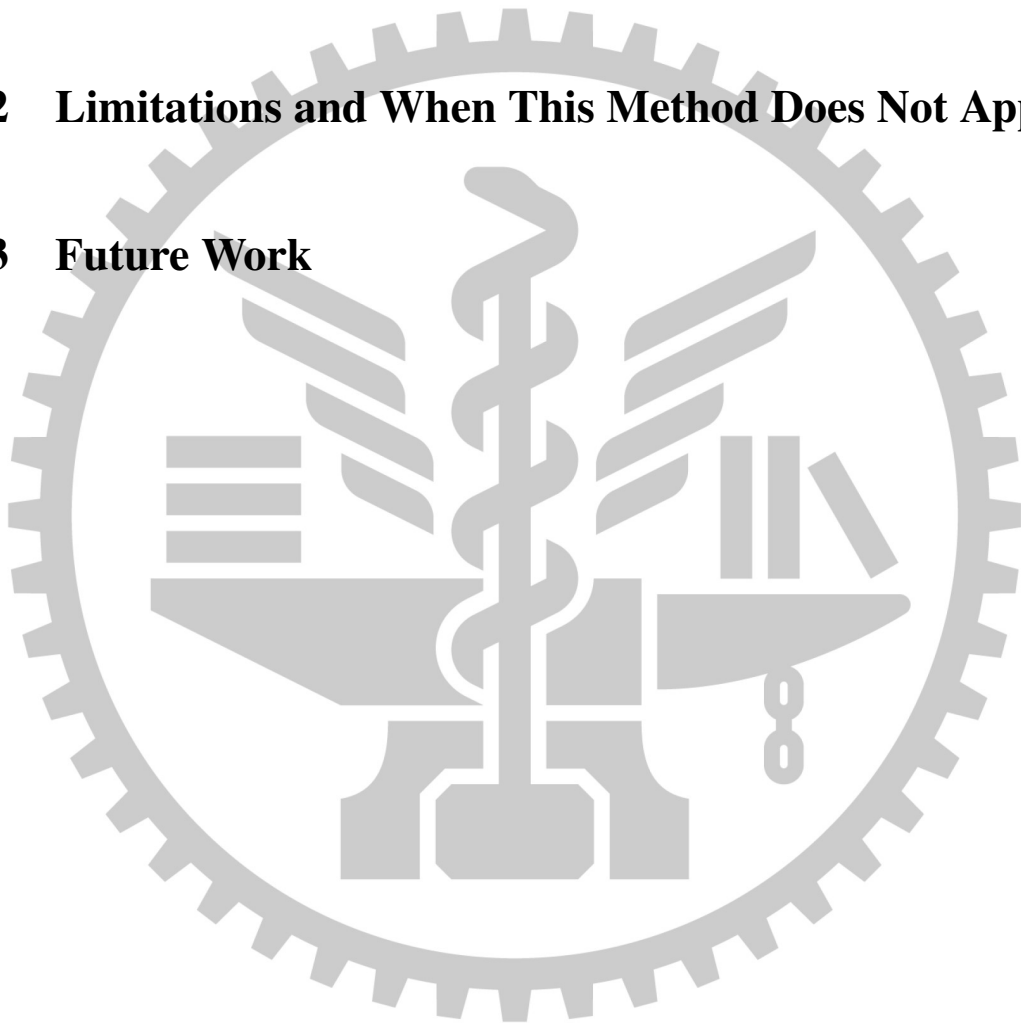
Status: implementation-backed for T1, T2, T5, T6, T7, T9, T10, T12; normative for T3, T4, T8, T11, T13. The items marked implementation-backed each name a measurement in this repository. The others are stated positions or unrun experiments, and are labelled so that no reader takes an intention for a result.

Chapter 6. Conclusions

6.1 Conclusion

6.2 Limitations and When This Method Does Not Apply

6.3 Future Work



References

